

계좌이체 뱅킹 서비스

PostgreSQL을 활용한 웹 서비스 개발 프로젝트

목차

1. 간단한 소개

- 1) 사용된 기술 스택
- 2) 사이트 소개
- 3) API 설계

2. 서비스 동작 과정

웹 서비스가 DBMS와 어떻게 연동되어 동작하는지

3. 백엔드의 데이터 정의 및 활용

- 1) 릴레이션
- 2) 쿼리
- 3) 트랜잭션

4. 퍼포먼스 테스트

- 1) 테스트 환경 세팅
- 2) 결과 해석

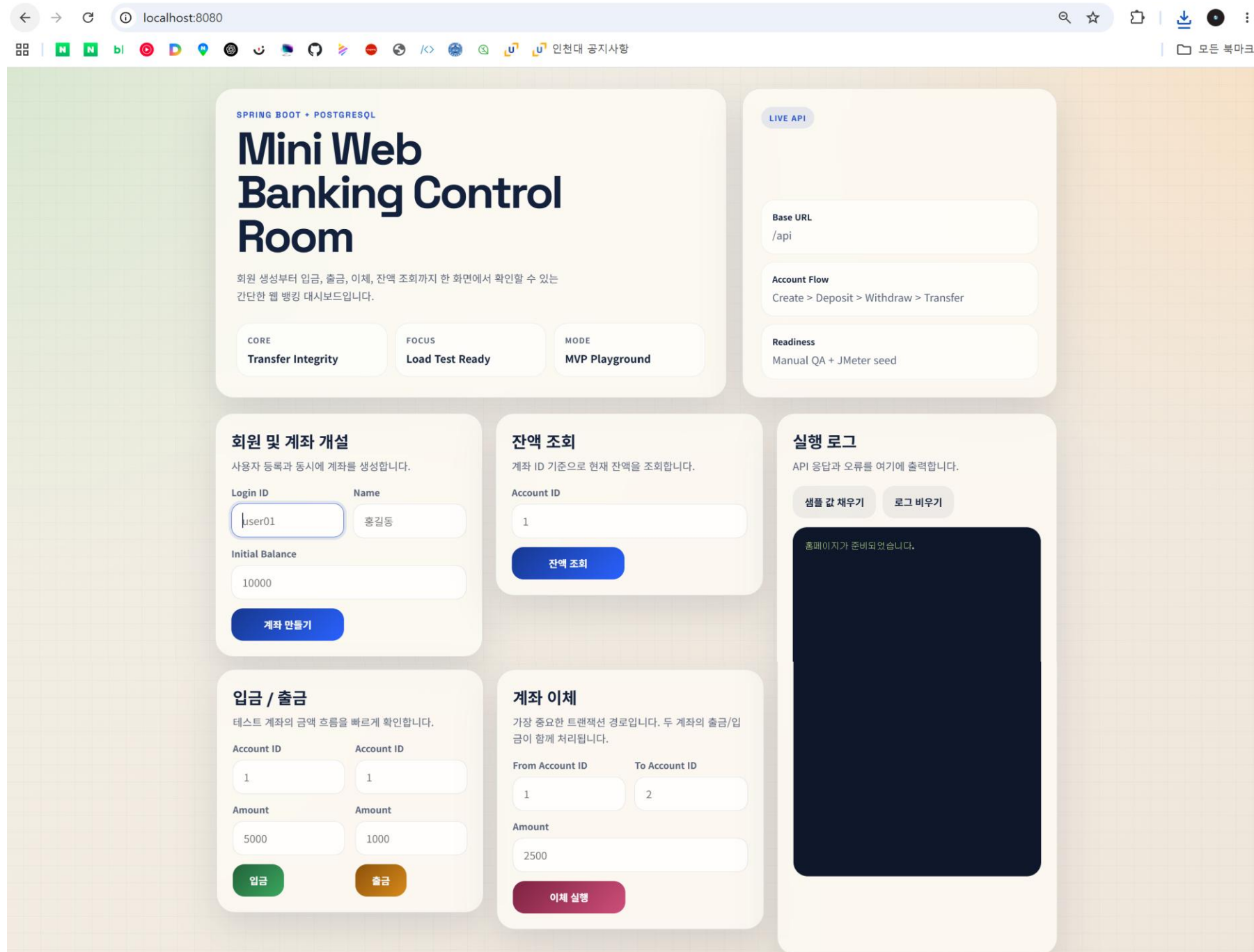
1. 간단한 소개



1) 사용된 기술 스택



2) 사이트 소개



3) API 설계

기능	HTTP Method	URL	요청 파라미터 (Request)	응답 데이터 (Response)	성공 코드
사용자 등록 & 계좌 개설	POST	/api/users	• loginId • name • initialBalance	• userId • accountId • accountNumber	201 Created
계좌 잔액 조회	GET	/api/accounts/{accountId}/balance	• accountId	• accountId • accountNumber • balance	200 OK
계좌 입금	POST	/api/accounts/{accountId}/deposit	• accountId • amount	• accountId • accountNumber • balance	200 OK
계좌 출금	POST	/api/accounts/{accountId}/withdraw	• accountId • amount	• accountId • accountNumber • balance	200 OK
계좌 이체 (송금)	POST	/api/transfers	• fromAccountId • toAccountId • amount	• transferId • fromAccountId • toAccountId • amount	201 Created

2. 서비스 동작 과정

버튼 클릭 - index.html

localhost:8080

인천대 공지사항

잔액 조회

계좌 만들기

출금

계좌의 금액 흐름을 빠르게 확인합니다.

ID

Account ID

1

Amount

1000

출금

계좌 이체

가장 중요한 트랜잭션 경로입니다. 두 계좌의 출금/입금이 함께 처리됩니다.

From Account ID

To Account ID

1

2

Amount

2500

이체 실행

```
</> index.html A X
src > main > resources > static > </> index.html > ...
2   <html lang="ko">
12  <body>
13  <div class="page-shell">
56  <main class="dashboard">
124
125  <section class="panel">
126  <div class="panel-head">
127    <h2>계좌 이체</h2>
128    <p>가장 중요한 트랜잭션 경로입니다. 두 계좌의 출금/입금이 함께 처리됩니다.</p>
129  </div>
130  <form id="transfer-form" class="form-grid">
131    <label>
132      <span>From Account ID</span>
133      <input type="number" name="fromAccountId" min="1" placeholder="1" required>
134    </label>
135    <label>
136      <span>To Account ID</span>
137      <input type="number" name="toAccountId" min="1" placeholder="2" required>
138    </label>
139    <label class="full-span">
140      <span>Amount</span>
141      <input type="number" name="amount" min="0.01" step="0.01" placeholder="2500" required>
142    </label>
143    <button type="submit" class="accent-rose">이체 실행</button>
144  </form>
145  </section>
```

사이트에서 정보를 입력한 후 이체 실행 버튼을 누르면 html에서 버튼 클릭을 감지하고 정보를 app.js로 넘긴다.

app.js

```
JS app.js A X
src > main > resources > static > JS app.js > ...
102   });
103
104   handleSubmit("transfer-form", async (form) => {
105     const payload = numberPayload(formDataToObject(form));
106     const result = await requestJson("/api/transfers", {
107       method: "POST",
108       body: JSON.stringify(payload)
109     });
110
111     appendLog("TRANSFER", result);
112   });
113
```

App.js는 사이트에서 받은 정보를 서버에게 보내기 위해 **Post /api/transfers** 라는 HTTP 요청을 보낸다.

이때 사이트에 입력한 정보들은 JSON 형태로 바디에 담아서 보내진다.

Controller

```
BankingController.java A X
src > main > java > com > Dweb > webBanking > controller > BankingController.java
22 import java.util.UUID;
23
24 @RestController
25 @RequestMapping("/api")
26 @RequiredArgsConstructor
27 public class BankingController {
28
29     private final AccountService accountService;
30     private final TransferService transferService;
31
32     @PostMapping("/users")
33     @ResponseStatus(HttpStatus.CREATED)
34     public CreateUserAccountResponse createUserWithAccount(@Valid @RequestBody CreateUserAccountRequest request) {
35         return accountService.createUserWithAccount(request);
36     }
37
38     @GetMapping("/accounts/{accountId}/balance")
39     public BalanceResponse getBalance(@PathVariable Long accountId) {
40         return accountService.getBalance(accountId);
41     }
42
43     @PostMapping("/accounts/{accountId}/deposit")
44     public BalanceResponse deposit(@PathVariable Long accountId, @Valid @RequestBody AmountRequest request) {
45         return accountService.deposit(accountId, request.amount());
46     }
47
48     @PostMapping("/accounts/{accountId}/withdraw")
49     public BalanceResponse withdraw(@PathVariable Long accountId, @Valid @RequestBody AmountRequest request) {
50         return accountService.withdraw(accountId, request.amount());
51     }
52
53     @PostMapping("/transfers")
54     @ResponseStatus(HttpStatus.CREATED)
55     public TransferResponse transfer(@Valid @RequestBody TransferRequest request) {
56         UUID transferId = transferService.transfer(
57             request.fromAccountId(),
58             request.toAccountId(),
59             request.amount()
60         );
61
62         return new TransferResponse(
63             transferId,
64             request.fromAccountId(),
65             request.toAccountId(),
66             request.amount()
67         );
68     }
69 }
```

서버는 컨트롤러에서 Post
/api/transfers 요청을 먼저 분류한
다.

성공적으로 요청을 마치면 201 코드로
응답한다.

TransferRequest라는 DTO 객체
로 정보를 담아서 Service에게 넘겨준
다.

Service

```
TransferService.java A X
src > main > java > com > Dweb > webBanking > service > TransferService.java
21 import java.util.stream.Collectors;
22
23 @Service
24 @RequiredArgsConstructor
25 public class TransferService {
26
27     private final AccountRepository accountRepository;
28     private final BankTransactionRepository transactionRepository;
29
30     @Transactional(isolation = Isolation.READ_COMMITTED)
31     public UUID transfer(Long fromAccountId, Long toAccountId, BigDecimal amount) {
32         if (fromAccountId.equals(toAccountId)) {
33             throw new IllegalArgumentException("같은 계좌로 이체할 수 없습니다.");
34         }
35         if (amount == null || amount.signum() <= 0) {
36             throw new IllegalArgumentException("이체 금액은 0보다 커야 합니다.");
37         }
38
39         List<Long> lockIds = List.of(
40             Math.min(fromAccountId, toAccountId),
41             Math.max(fromAccountId, toAccountId)
42         );
43
44         List<Account> locked = accountRepository.findAllByIdForUpdate(lockIds);
45         if (locked.size() != 2) {
46             throw new NoSuchElementException("계좌를 찾을 수 없습니다.");
47         }
48
49         Map<Long, Account> accountMap = locked.stream()
50             .collect(Collectors.toMap(Account::getId, Function.identity()));
51
52         Account from = accountMap.get(fromAccountId);
53         Account to = accountMap.get(toAccountId);
54     }
}
```

Service는 실제 비즈니스 로직을 담당한다.

계좌이체는 원자적으로 수행된다.

비즈니스적으로 발생할 가능성이 있는 예외들을 처리해준다.

Service

```
TransferService.java A X
src > main > java > com > Dweb > webBanking > service > TransferService.java
25 public class TransferService {
31     public UUID transfer(Long fromAccountId, Long toAccountId, BigDecimal amount) {
63         LocalDateTime now = LocalDateTime.now();
64
65         transactionRepository.save(BankTransaction.builder()
66             .account(from)
67             .txType(TransactionType.TRANSFER_OUT)
68             .amount(amount)
69             .balanceAfter(from.getBalance())
70             .counterpartyAccountId(to.getId())
71             .transferId(transferId)
72             .description("계좌이체 출금")
73             .createdAt(now)
74             .build());
75
76         transactionRepository.save(BankTransaction.builder()
77             .account(to)
78             .txType(TransactionType.TRANSFER_IN)
79             .amount(amount)
80             .balanceAfter(to.getBalance())
81             .counterpartyAccountId(from.getId())
82             .transferId(transferId)
83             .description("계좌이체 입금")
84             .createdAt(now)
85             .build());
86
87         return transferId;
88     }
89 }
90
```

비즈니스 로직을 모두 처리한 후 계좌이체를 DB에 반영하기 위해 Repository의 메서드를 호출한다.

Repository

BankTransactionRepository.java M X

src > main > java > com > Dweb > webBanking > repository > BankTransactionRepository.java

```
1 package com.Dweb.webBanking.repository;
2
3 import com.Dweb.webBanking.domain.BankTransaction;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface BankTransactionRepository extends JpaRepository<BankTransaction, Long> {
9 }
```

스프링 데이터 JPA를 사용하면 인터페이스를 정의하는 것만으로도 save, findById 등의 메서드들을 사용할 수 있다.

Save라는 메서드를 통해 DB에 값을 저장한다.

실제로는 쿼리가 일시적으로 쌓여 있다가 flush되면 쿼리들은 Hibernate를 통해 실제 쿼리문이 작성된다.

생성된 진짜 쿼리는 JDBC 드라이버를 통해 커넥션 통로를 타고 PostgreSQL DB로 전송된다.

DB로부터 성공 신호를 받으면 사용했던 커넥션은 다시 커넥션 풀에게 반납한다.

성공 - 응답

The image shows a web application interface on the left and a browser's network console on the right. The web application has several sections: 'CORE' with 'Transfer Integrity', 'Load Test Ready', and 'MVP Playground'; 'Readiness' with 'Manual QA + JMeter seed'; '회원 및 계좌 개설' (Member and Account Opening) with fields for Login ID, Name, Initial Balance, and a '계좌 만들기' button; '잔액 조회' (Balance Inquiry) with an 'Account ID' field and a '잔액 조회' button; '입금 / 출금' (Deposit / Withdrawal) with fields for Account ID, Amount, and buttons for '입금' and '출금'; '계좌 이체' (Account Transfer) with fields for From Account ID, To Account ID, Amount, and an '이체 실행' button; and '실행 로그' (Execution Log) with buttons for '샘플 값 채우기' and '로그 비우기'. The network console on the right shows a list of requests, with the first one selected. The selected request is a POST to 'http://localhost:8080/api/transfers' with a status code of 201 Created. A red box highlights the status code and the URL. A red arrow points from the status code to the text below.

간단한 웹 뱅킹 대시보드입니다.

Create > Deposit > Withdraw > Transfer

CORE
Transfer Integrity

FOCUS
Load Test Ready

MODE
MVP Playground

Readiness
Manual QA + JMeter seed

회원 및 계좌 개설
사용자 등록과 동시에 계좌를 생성합니다.

Login ID
user01

Name
홍길동

Initial Balance
10000

계좌 만들기

잔액 조회
계좌 ID 기준으로 현재 잔액을 조회합니다.

Account ID
1

잔액 조회

입금 / 출금
테스트 계좌의 금액 흐름을 빠르게 확인합니다.

Account ID
1

Account ID
1

Amount
5000

Amount
1000

입금

출금

계좌 이체
가장 중요한 트랜잭션 경로입니다. 두 계좌의 출금/입금이 함께 처리됩니다.

From Account ID
1

To Account ID
2

Amount
2500

이체 실행

실행 로그
API 응답과 오류를 여기에 출력합니다.

샘플 값 채우기

로그 비우기

[20시 56분 11초] TRANSFER

```
{
  "transferId": "679b5bdb-69a9-47e4-9ae4-b678e32a77ad",
  "fromAccountId": 1,
  "toAccountId": 2,
  "amount": 2500
}
```

홈페이지가 준비되었습니다.

1 / 18 requests | 0.3 kB / 0.9 kB t | Response headers (5)

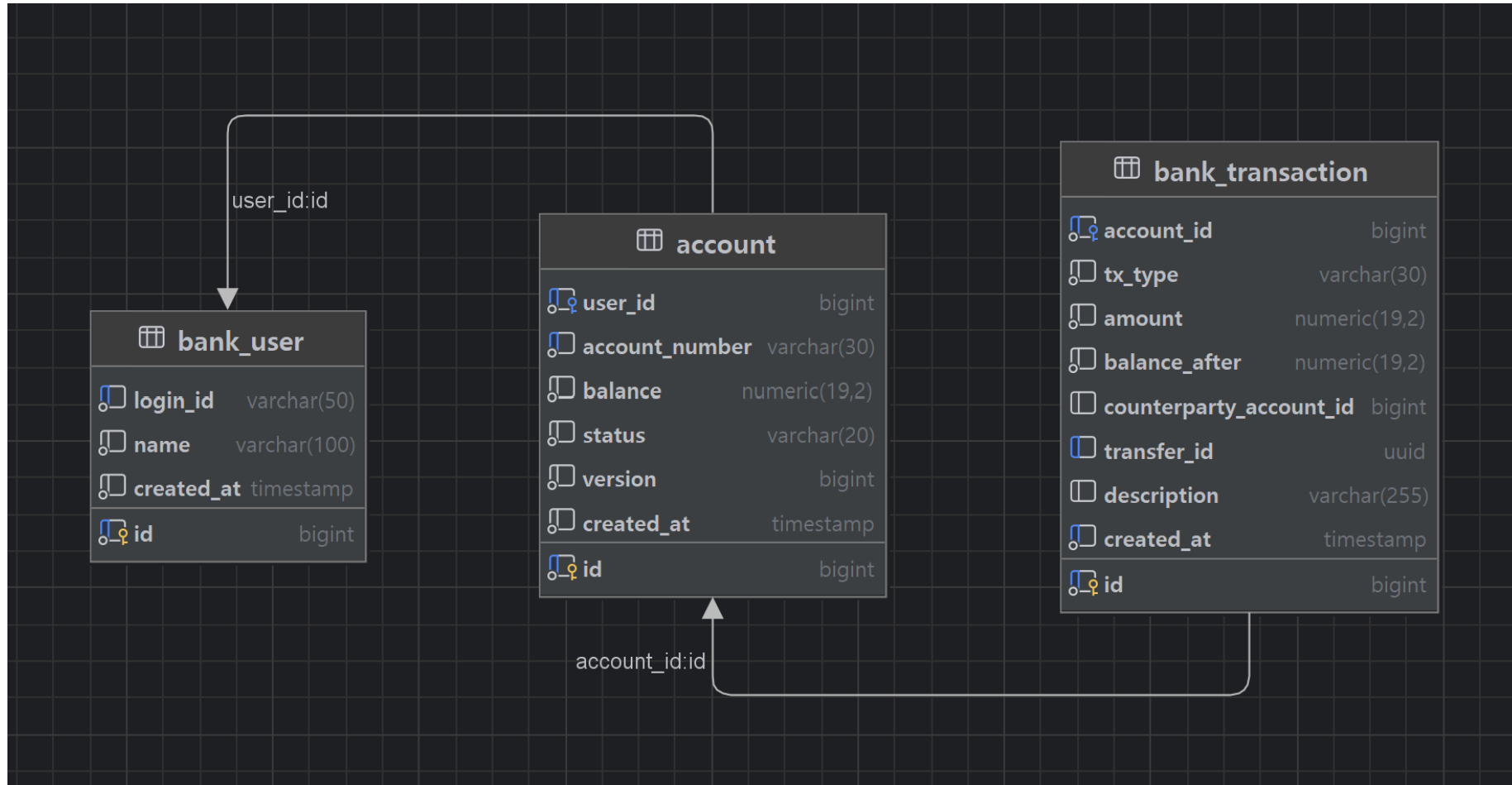
GET http://localhost:8080/favicon.ico 500 (Internal Server Error)

http://localhost:8080/api/transfers
POST
201 Created
[::1]:8080

모두 성공적으로 수행이 되면 201 코드로 클라이언트에게 응답한다.

3. 백엔드의 데이터 정의 및 활용

ERD 다이어그램



1) 릴레이션
In domain

```
22 @Entity
23 @Table(name = "account")
24 @Getter
25 @NoArgsConstructor(access = AccessLevel.PROTECTED)
26 public class Account {
27     @Id
28     @GeneratedValue(strategy = GenerationType.IDENTITY)
29     private Long id;
30
31     @ManyToOne(fetch = FetchType.LAZY, optional = false)
32     @JoinColumn(name = "user_id")
33     private BankUser user;
34
35     @Column(nullable = false, unique = true, length = 30)
36     private String accountNumber;
37
38     @Column(nullable = false, precision = 19, scale = 2)
39     private BigDecimal balance = BigDecimal.ZERO;
40
41     @Enumerated(EnumType.STRING)
42     @Column(nullable = false, length = 20)
43     private AccountStatus status = AccountStatus.ACTIVE;
44
45     @Version
46     private Long version;
47
48     @Column(nullable = false)
49     private LocalDateTime createdAt = LocalDateTime.now();
50
51     public static Account create(BankUser user, String accountNumber, BigDecimal initialBalance) {
52         Account account = new Account();
53         account.user = user;
54         account.accountNumber = accountNumber;
55         account.balance = initialBalance;
56         return account;
57     }
58
59     public void deposit(BigDecimal amount) {
60         this.balance = this.balance.add(amount);
61     }
62
63     public void withdraw(BigDecimal amount) {
64         if (this.balance.compareTo(amount) < 0) {
65             throw new IllegalStateException("잔액이 부족합니다.");
66         }
67         this.balance = this.balance.subtract(amount);
68     }
69 }
70
```

JPA는 엔티티를 통해 릴레이션을 설계

AUTO INCREMENT PRIMARY KEY

BankUser 릴레이션과 N:1 관계
user_id는 FK

NOT NULL

필드는 릴레이션의 속성과 연결

회원 릴레이션

```
BankUser.java A X
src > main > java > com > Dweb > webBanking > domain > BankUser.java
14
15 @Entity
16 @Table(name = "bank_user")
17 @Getter
18 @NoArgsConstructor(access = AccessLevel.PROTECTED)
19 public class BankUser {
20     @Id
21     @GeneratedValue(strategy = GenerationType.IDENTITY)
22     private Long id;
23
24     @Column(nullable = false, unique = true, length = 50)
25     private String loginId;
26
27     @Column(nullable = false, length = 100)
28     private String name;
29
30     @Column(nullable = false)
31     private LocalDateTime createdAt = LocalDateTime.now();
32
33     public static BankUser create(String loginId, String name) {
34         BankUser user = new BankUser();
35         user.loginId = loginId;
36         user.name = name;
37         return user;
38     }
39 }
```

UNIQUE 제약조건

계좌 이체 릴레이션

```
BankTransaction.java A X
C:\Users\wjdxo\Documents\web banking project\src\main\java\com\Dweb\webBanking\domain\BankTransaction.java (preview)
10 @Table(name = "bank_transaction")
11 @Getter @Builder
12 @NoArgsConstructor @AllArgsConstructor
13 public class BankTransaction {
14     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     @ManyToOne(fetch = FetchType.LAZY, optional = false)
18     @JoinColumn(name = "account_id")
19     private Account account;
20
21     @Enumerated(EnumType.STRING)
22     @Column(nullable = false, length = 30)
23     private TransactionType txType;
24
25     @Column(nullable = false, precision = 19, scale = 2)
26     private BigDecimal amount;
27
28     @Column(nullable = false, precision = 19, scale = 2)
29     private BigDecimal balanceAfter;
30
31     private Long counterpartyAccountId;
32
33     private UUID transferId;
34
35     private String description;
36
37     @Column(nullable = false)
38     private LocalDateTime createdAt;
39 }
```

Enum은 실제로 숫자로 기록 -> 확인 어려움
따라서 DB에서 문자열로 저장될 수 있도록 함

2) 쿼리 In Repository

```

BankTransactionRepository.java M X
src > main > java > com > Dweb > webBanking > repository > BankTransactionRepository.java
6
7 | @Repository
8 public interface BankTransactionRepository extends JpaRepository<BankTransaction, Long> {
9 }

```

스프링 데이터 JPA를 사용하면 인터페이스 정의만으로 기본적인 쿼리를 사용할 수 있다.
Ex) save, findById, findAll, deleteById

```

BankUserRepository.java M X
src > main > java > com > Dweb > webBanking > repository > BankUserRepository.java
9 | @Repository
10 public interface BankUserRepository extends JpaRepository<BankUser, Long> {
11     Optional<BankUser> findByLoginId(String loginId);
12 }
13

```

findBy[필드명] 이런 식으로 메서드를 정의해두면 JPA가 알아서 구현해준다.

```
AccountRepository.java M X
src > main > java > com > Dweb > webBanking > repository > AccountRepository.java
12 @Repository
13 public interface AccountRepository extends JpaRepository<Account, Long> {
14
15     Optional<Account> findByAccountNumber(String accountNumber);
16
17     @Lock(LockModeType.PESSIMISTIC_WRITE)
18     @Query("select a from Account a where a.id = :accountId")
19     Optional<Account> findByIdForUpdate(@Param("accountId") Long accountId);
20
21     @Lock(LockModeType.PESSIMISTIC_WRITE)
22     @Query("select a from Account a where a.id in :ids order by a.id")
23     List<Account> findAllByIdForUpdate(@Param("ids") List<Long> ids);
24 }
25
```

JPQL이라는 쿼리 언어를 사용하여 더 복잡한 쿼리를 처리할 수 있다.

비관적 락을 적용하는 애노테이션으로 쿼리를 보낼 때 FOR UPDATE를 붙여서 보내준다.

3) 트랜잭션 In Service

```
src > main > java > com > Dweb > webBanking > service > TransferService.java
22
23 @Service
24 @RequiredArgsConstructor
25 public class TransferService {
26
27     private final AccountRepository accountRepository;
28     private final BankTransactionRepository transactionRepository;
29
30     @Transactional(isolation = Isolation.READ_COMMITTED)
31     public UUID transfer(Long fromAccountId, Long toAccountId, BigDecimal amount) {
32         if (fromAccountId.equals(toAccountId)) {
33             throw new IllegalArgumentException("같은 계좌로 이체할 수 없습니다.");
34         }
35         if (amount == null || amount.signum() <= 0) {
36             throw new IllegalArgumentException("이체 금액은 0보다 커야 합니다.");
37         }
38
39         List<Long> lockIds = List.of(
40             Math.min(fromAccountId, toAccountId),
41             Math.max(fromAccountId, toAccountId)
42         );
43
44         List<Account> locked = accountRepository.findAllByIdForUpdate(lockIds);
45         if (locked.size() != 2) {
46             throw new NoSuchElementException("계좌를 찾을 수 없습니다.");
47         }
48
49         Map<Long, Account> accountMap = locked.stream()
50             .collect(Collectors.toMap(Account::getId, Function.identity()));
51
52         Account from = accountMap.get(fromAccountId);
53         Account to = accountMap.get(toAccountId);
54
55         if (from.getStatus() != AccountStatus.ACTIVE || to.getStatus() != AccountStatus.ACTIVE) {
56             throw new IllegalStateException("비활성 계좌는 이체할 수 없습니다.");
57         }
58
59         from.withdraw(amount);
60         to.deposit(amount);
61
62         UUID transferId = UUID.randomUUID();
63         LocalDateTime now = LocalDateTime.now();

```

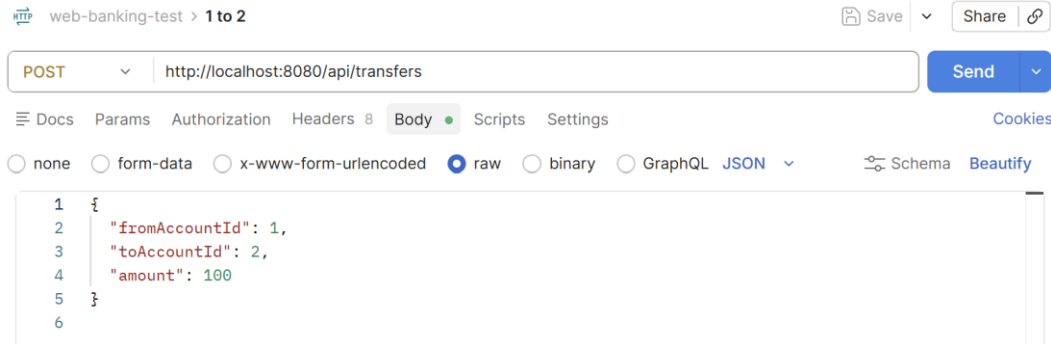
애노테이션을 통해 트랜잭션 표시
격리 레벨도 설정 가능

데드락을 방지하기 위해 항상 계좌
id가 더 낮은 계좌가 락을 선점하도
록 설계

이체를 위해 잔고를 조회할 때 row
락 설정

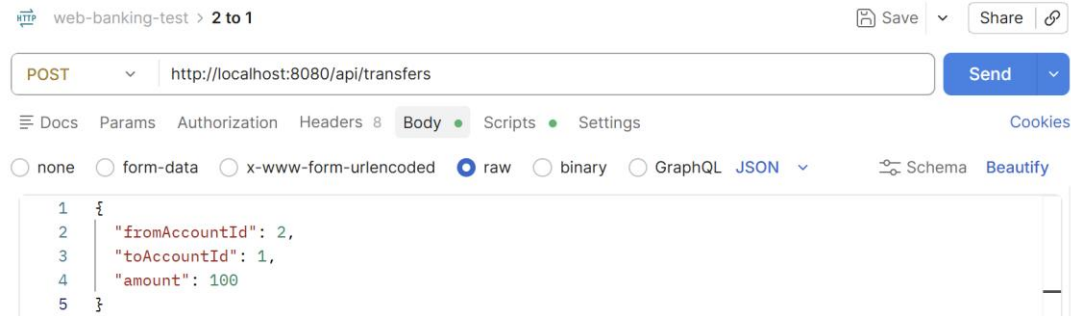
4. 퍼포먼스 테스트

1) Postman - 테스트 환경 세팅



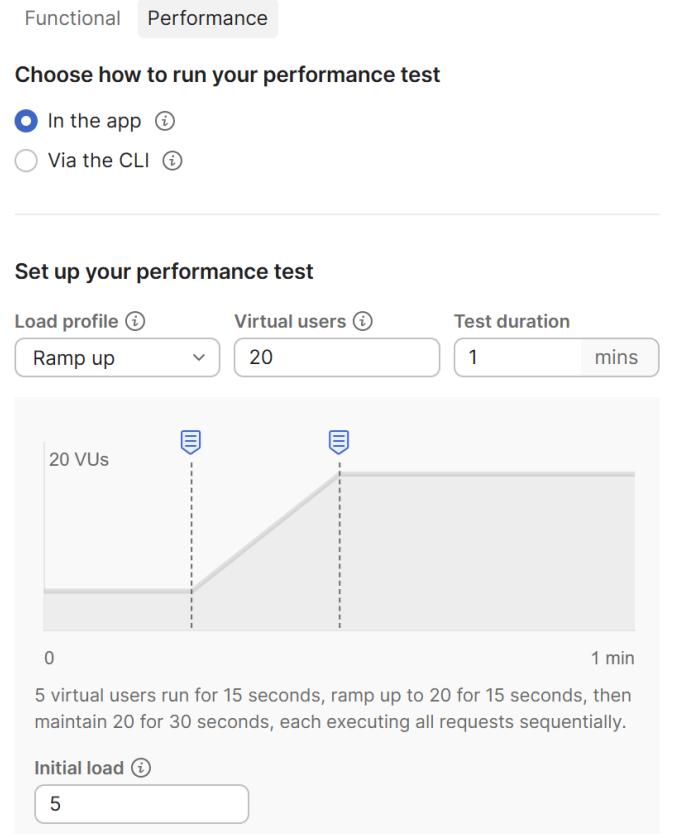
<Request #1>

계정 1 -> 계정 2 : 100원 계좌이체



<Request #2>

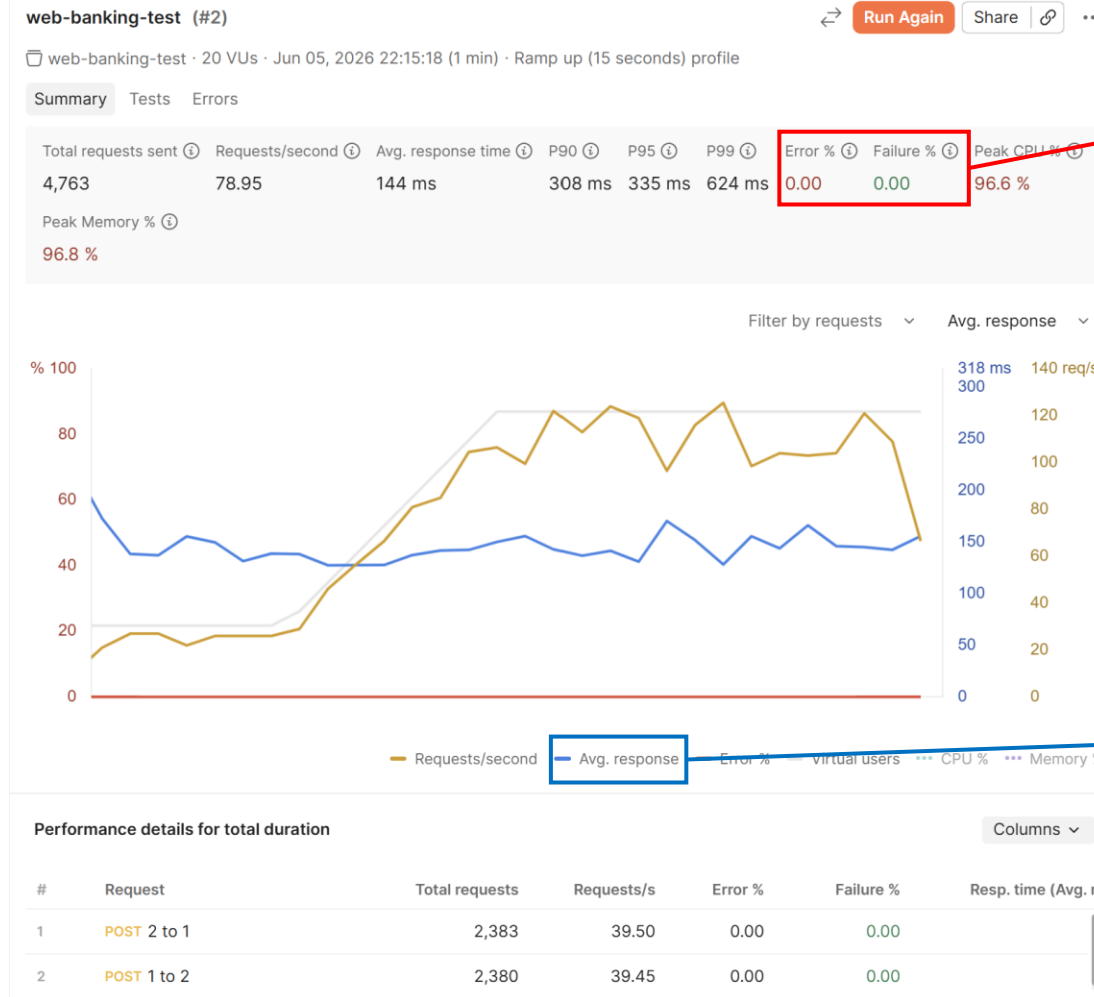
계정 2 -> 계정 1 : 100원 계좌이체



<테스트 세팅>

- 유저 수가 서서히 증가하는 Ramp up 방식
- 최대 유저 수 20명
- 테스트 시간 1분

2) Postman - 결과 해석



오류가 발생하지 않았음
-> 데드락 문제가 발생하지 않음

평균 응답 시간이 안정적
-> 안전하면서도 준수한 트래픽 처리